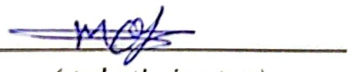
Moscow Institute of Physics and Technology
(national research university)
Phystech School of Applied Mathematics and Informatics
Chair of Discrete Mathematics

Major of study: Information Science and Computer Engineering
Specialization: Computer Science/Информатика

CLASSIFICATION OF EEG DATA WITH FEDERATED LEARNING/ПРИМЕНЕНИЕ ФЕДЕРАТИВНОГО ОБУЧЕНИЯ ДЛЯ ЗАДАЧИ КЛАССИФИКАЦИИ ЭЛЕКТРОЭНЦЕФАЛОГРАММ

(Bachelor)

Student:
Mostafa Eid



(student's signeture)

Scientific supervisor:
Beznosikov Aleksandr Nikolaevich,
канд. физ.-мат. наук



(supervisor's signature)

Moscow 2024

Abstract

Privacy-concerned approaches for training machine learning models have been rapidly developing in recent years. Benchmarks and comparative studies have been explored and set on tabular data, while other forms of data are barely explored. Motivated by other studies that introduce a standard on how to work with tabular data, In this thesis we implement FedAvg and SCAFFOLD, which are two of the most robust federated learning algorithms on EEG data. We also see how the distribution of the data and its homogeneity affect the performance of the model.

Our methods were able to achieve a kappa score of 0.70 which is substantial and is on the same level as the kappa score which is achieved by the centralized approach. One of the main challenges of federated learning is the unbalanced distribution of the data between the clients. We tried to address this issue by using the SCAFFOLD algorithm with a very low learning rate which resulted in a kappa score of 0.3 and AUROC 0.69 score.

Table of Contents

1. Introduction..... 1

Artificial Intelligence and Machine Learning..... 1

Data In Machine Learning..... 1

Federated Learning (FL)..... 2

2. Background..... 3

Classification..... 3

K-nearest Neighbors (KNN)..... 3

Naive Bayes (NB)..... 4

Logistic Regression..... 5

Artificial Neural Networks (ANN)..... 5

Backpropagation..... 7

Activation Functions..... 8

Linear..... 8

Sigmoid..... 9

Hyperbolic Tangent (tanh)..... 9

Rectified Linear Unit (ReLU)..... 10

Leaky ReLU..... 10

Maxout..... 11

Gated Recurrent Unit (GRU)..... 12

Evaluating Machine Learning Models..... 12

Confusion Matrix..... 13

Accuracy..... 13

Precision..... 13

Recall..... 14

F1..... 14

KAPPA..... 14

Receiver operating characteristics (ROC)..... 15

Federated Learning..... 16

Federated Learning Properties and Suppositions..... 17

Non-IID Data..... 17

Categories of Federated Learning..... 20

Electroencephalography (EEG)..... 21

3. Related Work..... 23

4. Methods..... 25

The Datasets and Classes..... 25

Data preprocessing..... 26
Training..... 26
Model.....26
Federated Learning Algorithms.....27
5. Results.....31
6. Conclusions..... 36
Bibliography.....37

1.Introduction

Artificial Intelligence and Machine Learning

Artificial Intelligence (AI) is a broad term that doesn't have one specific definition and has been defined by many researchers throughout its development. In this thesis, we will use the following definition of AI systems "systems that display intelligent behavior by analyzing their environment and taking actions – with some degree of autonomy – to achieve specific goals" by the High-Level Expert Group on Artificial Intelligence (AI HLEG) of the European Commission (EC), as it provides a good balance containing all the applications of AI as we know them today, and the ones that we will see in the future. [1]

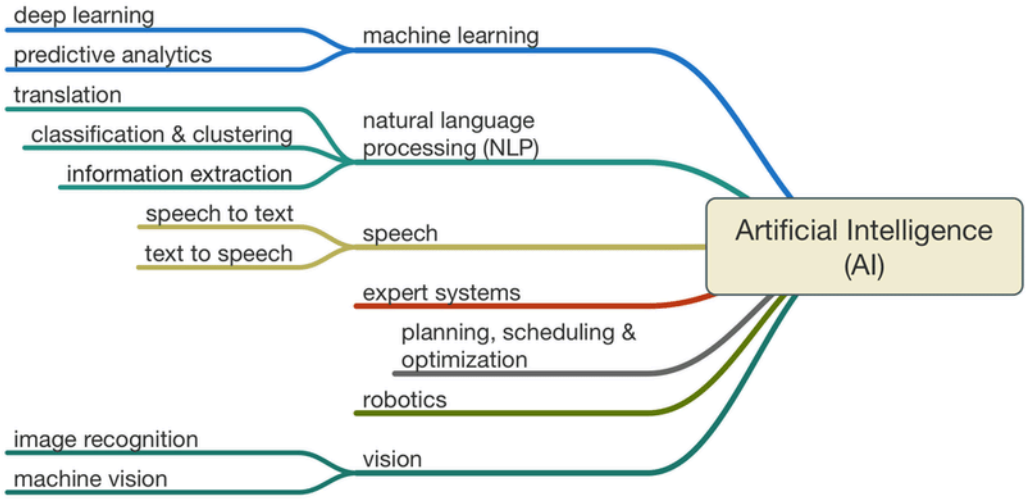


Figure 1 [2]

Emerging from the more traditional statistical modeling, Machine Learning (ML) is a branch of AI that covers a wide variety of algorithms that make predictions from a data set. These techniques can vary from complex neural networks to simpler ones like linear regression. The way we choose the algorithm depends on the type of data we have and the problem we are trying to solve. For example, the size of the dataset can play a vital role. [3]

Data In Machine Learning

We live in an era where we produce around 1.72 trillion photos a year, 54,000 each second, 3.3 million a minute, or 4.7 billion photos a day. [4]In other words, we produce a whopping amount of data on a daily basis, but not all of this data is available due to several reasons. One of the main

concerns to consider is how sensitive this data is, which can occur in important aspects such as healthcare or energy. In such cases, the data is usually anonymized to remove any personal information. [5]

There are several methods that are being used to keep the privacy of the data in ML such as Data anonymization, Differential privacy, Secure multi-party computation, and Homomorphic encryption. In 2018 the General Data Protection Regulation (GDPR) was applied in all European Union Countries, creating more restrictions and requirements on data collection, and focusing on six data protection principles. Thus ML algorithms faced a challenge and to survive these new challenges Federated Learning (FL) was born. [6]

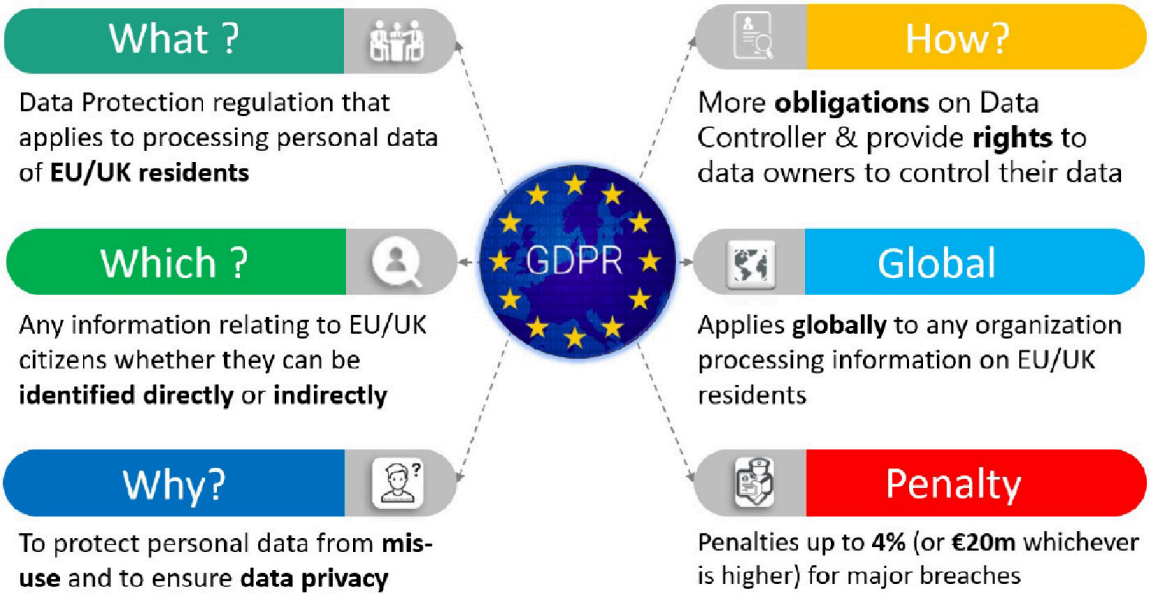


Figure 2 Summary of the GDPR [6]

Federated Learning (FL)

Federated Learning is a decentralized machine learning technique that doesn't require the data to be transferred to a central unit but rather to be kept on the users' devices. [7]

2.Background

Machine Learning has several types depending on the goal we are seeking to achieve and there are four major types:

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning

Supervised Learning: It works on finding and understanding relationships between input data and output data. One of the main tasks of supervised learning is classification and it is called supervised because the data used in the training is labeled or the relationships between the data is already determined. The most famous example of this task is the classification of hand-written numbers from 0 to 9 in the MNIST dataset. [8], [9]

Classification

According to [10], classification is a two-step process, where the first part is extracting the classifier or the model that illustrates the main classes in the given data, and the second step is determining whether the accuracy of the model is sufficient and based on that deciding if it can be used on new data.

The dataset on which the model is built is called a **training set** and it consists of data objects and the labels that belong to them. The new data over which we test the accuracy of the model is called a **test set** and it also has data objects with their corresponding labels but it is independent from the training set, meaning that the constructed model didn't see it before.

We are going to discuss a few main classification techniques and focus mainly on the usage of Neural Networks (NN) for classification as it is the method we use in this thesis.

K-nearest Neighbors (KNN)

This algorithm extends over the nearest neighbor idea, instead of using one sample it uses k-samples to determine the class of the new or examined sample. The nearest neighbor idea is very simple, the new or examined sample shares the same class with its closest sample. More formally, for a new sample X, the class for X is determined by finding the smallest value of the function g_i for i belonging to the set of classes.[11] $g_j(X) = \min g_i(\mathrm{X}) \quad i = 1, 2, \ldots, C$

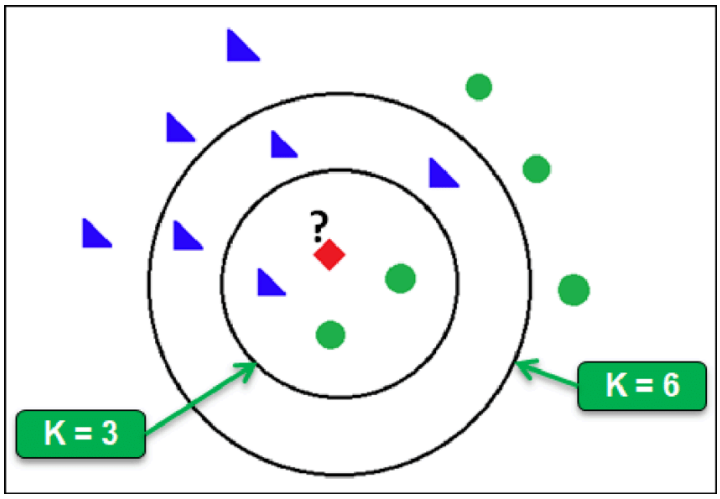


Figure 3 KNN explanation with k 3 and k=6 neighbors (blue, green, and red show distribution of class of data). [12]

Naive Bayes (NB)

To understand Naive Bayes, we first have to know Bayes' Rule which helpd calculate the posterior $P(A|B)$ from $P(B)$ which is called the prior, $P(B|A)$ the likelihood, and $P(A)$.

$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

Bayes' Rule could be easily derived from the principle of conditional probability in the following way: $P(B|A) = \frac{A \cap B}{P(A)}$ (multiply by $P(A)$) we get $P(B|A)P(A) = P(A \cap B)$ we also have $P(A|B) = \frac{P(A \cap B)}{P(A)}$ substitute it in the previous equation we get $P(B|A)P(A) = P(A|B)P(A)$ and finally $P(A|B) = \frac{P(B|A).P(A)}{P(B)}$

Naive Bayes (NB) [13], Is a learning algorithm based on Bayes' rule and the supposition that predictors are conditionally independent. Although this supposition doesn't always hold when put into practice, NB still shows relatively good accuracy. NB classifier's best feature is its efficiency as the training and the classification can be done in only one pass over the data.[14] [15]

Logistic Regression

We will only cover the base case of binary logistic regression, which describes a learning problem where the target data is categorical and depends on one or more variables. It can be represented in the following manner:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \ + \theta_n x_n$$

$$h(\hat{y}) = \frac{1}{1 + \epsilon^{-}\hat{y}}$$

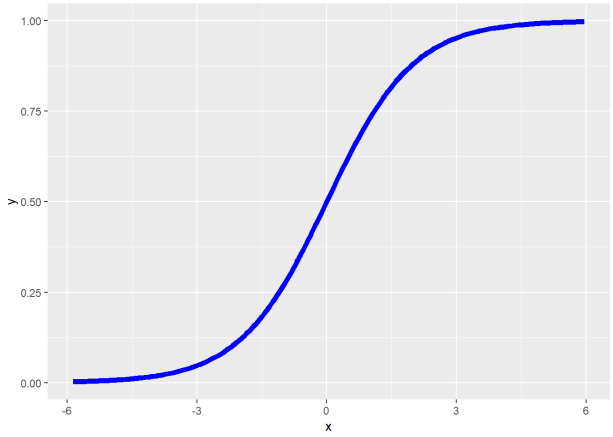


Figure 4 The shape of the curve of the sigmoid function

This function is called the logistic, logit, or sigmoid function, which has the shape of the letter S, and works on bounding the result between 0 and 1. [16]

Artificial Neural Networks (ANN)

Artificial Neural Networks (ANN) were inspired by our fascination on how the human brain functions and operates on huge amounts of inputs quickly and in parallel achieving several goals like motor control, perception and pattern recognition. [17] trying to mimic this structure we came up with artificial neurons and using many of these and several layers we created multilayer (deep) neural networks. [18] Figure 5 represents the general structure of a neural network with $x_0, \ldots, x_p$ representing the input data that the neurons receive either from an external system or from previous neurons it has connection with. $w = (w_0, , w_p)$ are the weights or more specifically the synaptic weights and their

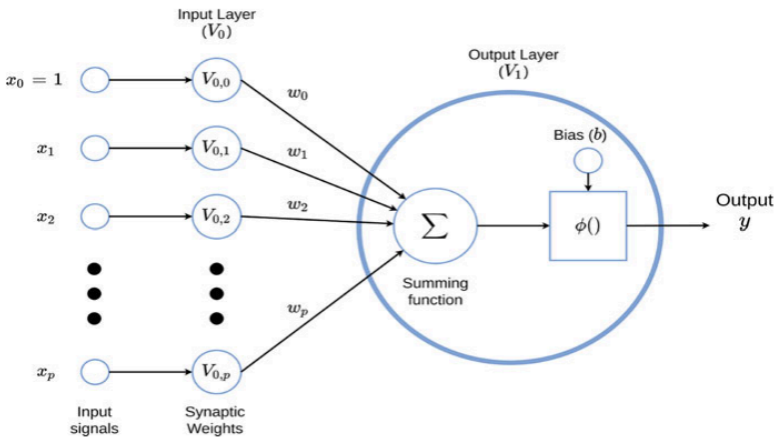


Figure 5 General artificial neural network model [18]

$$v_j = \sum_{j=1}^{p} w_{ij} x_j$$

role is to modify the inputs, all of the different values added together to give the net input which can be mathematically interpreted in the following way:

The net input v_j determines if the neuron is activated or not, and the activation depends on what is called an activation function. The output of the network results from this function applied on the net input $y_j = g(v_j)$ where g is the activation function. The unilayer ANN doesn't have the high capacity we are looking for, but rather the connection of many ANNs is where the power of this concept lies. It consists of an input layer, hidden layer/s and output layer, and the depth of the network is the number of layers except the input layer. [18]

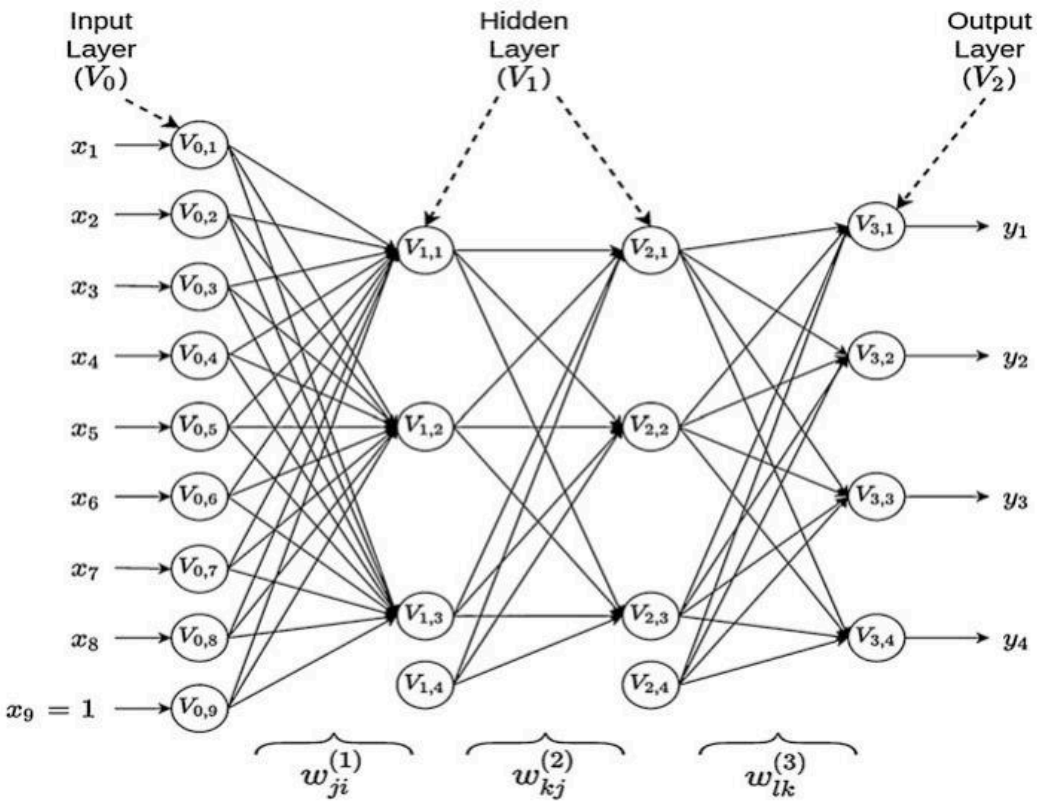


Figure 6 Artificial deep neural network with a feedforward neural network with eight input variables (x1, … ,x8), four output variables (y1, y2, y3, y4), and two hidden layers with three neurons each [18].

In Figure 6 the ANN has 2 hidden layers (V_1 , V_2), one input layer V_0 and V_3 which is the output layer. [18]

Input layer: this layer is the neurons that get the information directly from the outer source of the network, and for Figure 6 this would be $x_1, , x_9$. [19] The number of neurons in this layer is mostly the same as the number of the input features and typically this layer is followed by one or more hidden layers. In classical feed-forward ANNs, the first hidden layer is fully connected.[20]

Hidden Layer: These layers are what give multilayer ANNs the ability to model nonlinear functions and the number of hidden layers can be 0 or several layers. [19] The neurons in the hidden layers work as feature detectors, as during the learning process they little by little start to uncover the patterns that we have in the training data.[17]

Output Layer: This is the layer from which we receive the anticipated result from the model. The nature of the model's output might be probabilities (classification task) or real-valued output (regression task). [20]

Backpropagation

The learning of ANNs depends on a repetitive process of adjusting the weights and the biases of the network, and for this, we need what is called a loss function, that provides an estimation of the error rate between the output of the ANN and the real values or labels of the data we are using. Loss functions vary depending on the type of task we are working on. For example, if we are working on a classification problem, for a regression problem usually Mean Squared Error is used.

So, backpropagation depends mainly on the following steps [21], [22]:

1- On the samples that you are training the model on, calculate the activation function, then find the output. These values are usually called forward values.

2- Calculate the partial derivative of the Loss Function, which helps us know how much the error changes based on the weighted sum of inputs and includes the following substeps:

a- Evaluate the Error Term for the Final Layer.

b- Backpropagate the Error Terms for Hidden Layers.

c- Evaluate the Partial Derivatives of the Error.

3- Combine Individual Gradients and Update Weights.

Moreover, there are several points that can make a huge difference in the training process on which we should focus. The learning rate, which is a hyperparameter to be configured before the training is the rate at which the weights change during the training and plays an important role in the training process. Momentum can help mitigate the changes that might occur because of the changes in the

weights, and that is by adding a small portion of the last weight change to the new weight change [23].

Activation Functions

Activation functions determine when and should the neuron fire its information to the next layer. Simply, these functions decide if the neuron has the information to produce a correct prediction as an output of the ANN for a sample in the training set. The input of the activation function is a scalar which is the weight sum of the weights and biases and gives back a scalar output. These functions are what give the ANN its nonlinearity and that's why they are called non-linearities. We will now consider the most common activation functions. [16], [18]

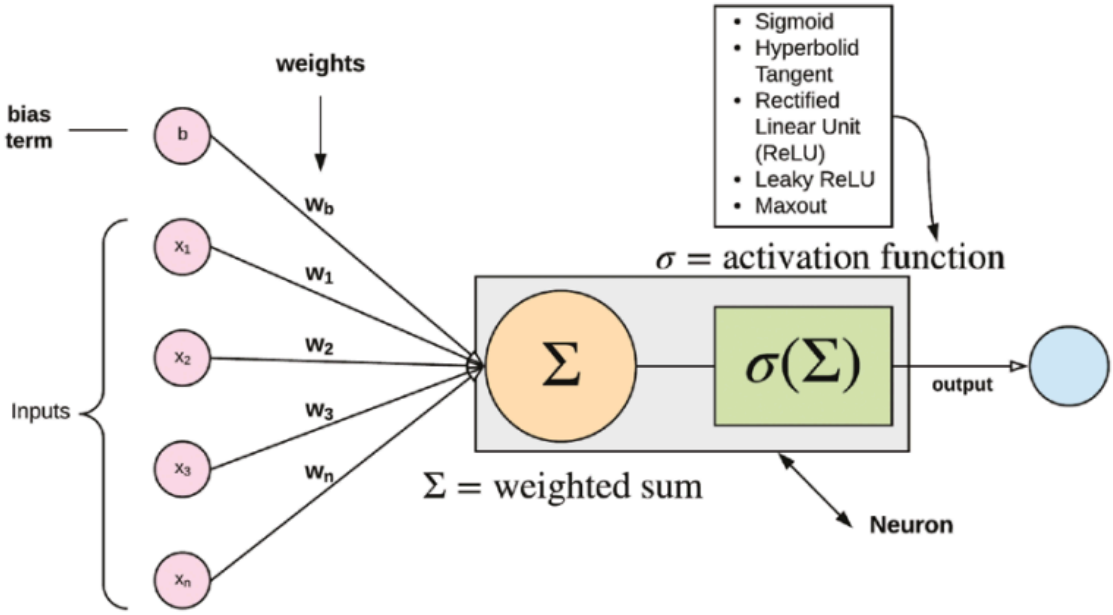


Figure 7 Activation function. Adapted from [16]

Linear

The linear function is a simple activation function and in Figure 8 we have the identity function. This function can be expressed as $f(x) = cx$ where x is the input and c is a weight or constant.

The problem with this function is that it can't give the neural network the ability to express non-linearities and usually the data is rarely linear in practice [24].

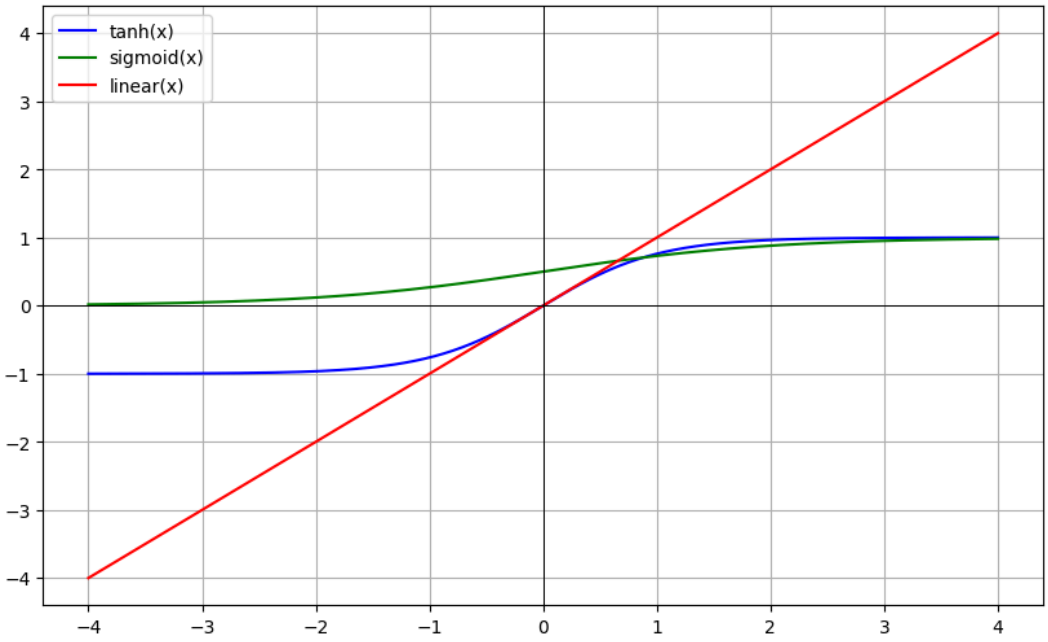


Figure 8 shows the graph of Linear, Logistic Sigmoid, and Tanh activation functions

Sigmoid

Is one of the most common activation functions to create Deep Neural Networks or ANNs for binary classification. The function has the following form $\sigma(x) = \frac{1}{1 + \ exp(-x)}$ which makes the most of its output between 0 and 1. One of its downsides is the tendency to get stuck which means that the learning process of the model won't improve due to the large value of the output of this activation function [18]. Another disadvantage of the sigmoid function is that it is not zero-centric which translates to poor convergence [25]. Lastly, during the process of optimization of the weights of the network in backpropagation, the gradients can get very large or very small, and the output of the activation near 1 or 0 as a result and after several multiplications during backpropagation these gradients either explode or vanish, which reflects badly on the training process of the network [16].

Hyperbolic Tangent (tanh)

This is a hyperbolic trigonometric function that can be expressed in a similar way we express the tangent as the ratio between the opposite and the adjacent sides of a right triangle, tanh is the hyperbolic sine function to the hyperbolic cosine function [20].

Tanh function: $$tanh(x) = \frac{sinh(x)}{cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivative of tanh function: $$\frac{d}{dx}tanh(x) = 1 - tanh^2(x)$$

What differentiates tanh from the sigmoid function is that first, it has a range from -1 to 1 so the negative values will have a negative output too and the values close to zero will have their output close to zero as well. The second thing that makes the tanh function preferable over the sigmoid function is that its derivative has a bigger value near zero. It is commonly used for binary classification tasks. Finally, and just like the sigmoid function, tanh suffers from the vanishing gradient problem [24].

Rectified Linear Unit (ReLU)

This function uses a threshold to activate and usually, if the value is less than zero then the output is zero otherwise it would act like the identity function and we would express it mathematically by the following expression: $f(x) = max(0, x)$ [24]. The ReLU has a significant advantage over the sigmoid and tanh functions by reducing the problem of the vanishing gradient [16].

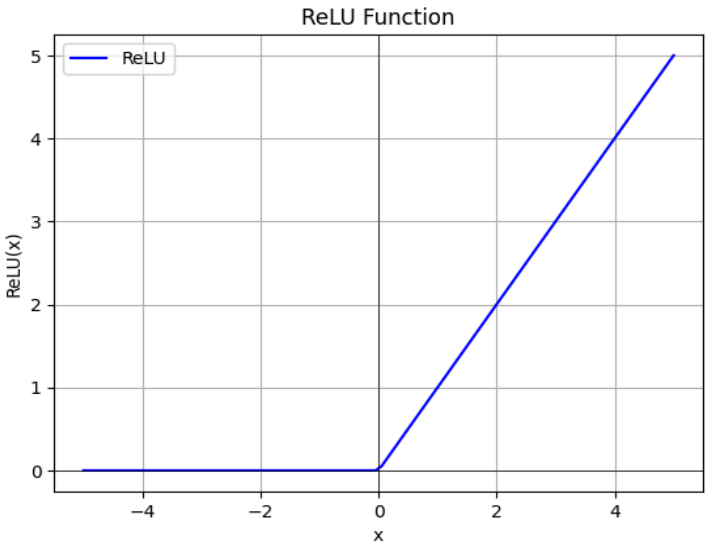


Figure 9 Rectified Linear Unit Function

One downside to ReLU is "dying" as these might learn big bias parameters in a way that they will always output the same value and as a result, the gradient will always be zero after this. This problem can be partially solved by setting a proper learning rate [26].

Leaky ReLU

Trying to fix the problem of "dying ReLU" and instead of having zero when x <= 0, Leaky ReLU will have a small positive number (e.g. 0.01) as slope. Thus, the function would become like this:

$$f(a,b) = \begin{cases} \text{x} & \text{if x>0} \\ 0.01\text{x} & \text{otherwise.} \end{cases}$$

The results of this form are not always consistent [26].

Maxout

Choosing the factor for the leaky ReLU can be hard, and this is why a more general form was created by the researchers. Maxout is a more general function that utilizes the effectiveness of ReLU and stays away from "dying ReLU" but this comes at the cost of increased computation because Maxout doubles the number of neurons in the neural network [16], [24]. It can be expressed mathematically in the following way: $f(x) = max(\vec{w_1} * \vec{x} + b_1, \vec{w_2} * \vec{x} + b_2)$ [24].

Function	**Comment**	**When to use?**
Sigmoid	Vulnerable to the problem of vanishing gradient and zigzagging during training because it is not zero-centric	Can be used in simulating logic gates, and can also be used for predicting probability-based output [27].
tanh	It is also susceptible to the vanishing gradient problem	Can be used in recurrent neural networks
ReLU	It is the most commonly used activation function for the hidden layers of neural networks. In some cases, it can be vulnerable to the "dying ReLU" problem	Should be the first choice
LReLU	Eases the problem of dying ReLU while maintaining all of its pros	Use it If you expect facing the "dying ReLU" problem
Maxout	Advanced Activation function, solves the problem of "dying ReLU". From a computational point of view, it has a very high cost	Could be tried if the neural network is very deep

Figure 10 Recommendation on when to use which activation function in deep neural networks adapted from [24] (Please take a look at the reference for the whole table.).

Gated Recurrent Unit (GRU)

In this chapter we will try to simply explain the work of the GRU which was first introduced by [28], and it is a special type of hidden units that is considered a variant of the Long short-term Memory (LSTM) unit but with simpler structure making it easier to implement and faster to compute.

The following equations express it:

This equation represents the *reset gate*:

$$r_t = \sigma(W_r.[h_{t-1}, x_t]) \qquad (1)$$

The next equation expresses the *update gate* and controls the amount of the update the GRU unit will have:

$$z_t = \sigma(W_z.[h_{t-1}, x_t]) \qquad (2)$$

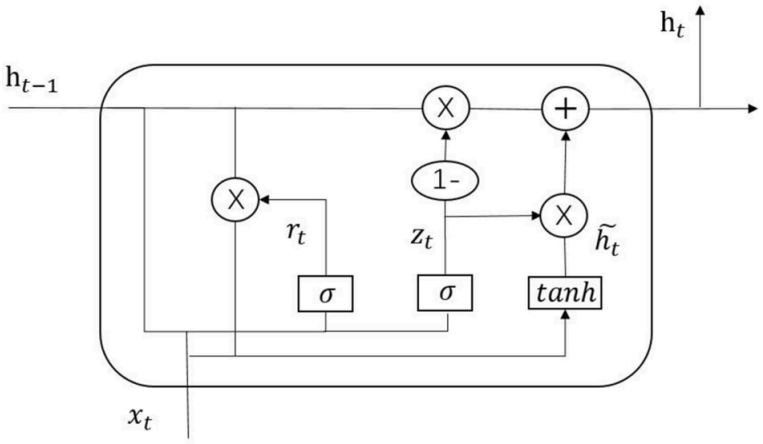


Figure 11 Gated Recurrent Network Diagram [29]

This equation is a new remember gate:

$$tildah_t = \tanh(W_r.[r_t \otimes h_{t-1}, x_t])$$

The hidden state of the GRU unit is computed using this function:

$$h_t = (1 - z_t) \otimes h_{t-1} + z_t \otimes h_t$$

We can see that like this if the *reset gate* has its value close to zero the hidden state will neglect the effect of the previous state and only consider the current input.

The square brackets are for concatenation and $\otimes$ for tensor products. σ is the sigmoid function which we covered in the part about activation functions and has an output between 0 and 1. The activation function $tanh$ was also covered in the previous section [28], [29].

Evaluating Machine Learning Models

To see how well or how bad a model is providing the right classification and after that to measure the value of the prediction is what is called evaluation of a model. This process can be straightforward

measuring only the portion the model got right, but in some situations, we might care more about a certain kind of the predictions being correct more than the others.

Confusion Matrix

A confusion matrix (Figure 12) is a table that consists of rows and columns representing the predicted outcomes and the real outcomes of a classification model. It is usually used to express and show how good or bad the performance of the model is [20].

We use the following number to help us answer these questions:

- True Positives
  - Correct prediction
  - Has a positive label
- False Positives
  - Wrong prediction
  - Has a negative label
- True negatives
  - Correct prediction
  - Has a negative label
- False negatives
  - Wrong prediction
  - Has a positive label

	P' (Predicted)	N' (Predicated)
P (Actual)	True Positive	False Negative
N (Actual)	False Positive	True Negative

Figure 12 The Confusion Matrix

False positives are called "type I error" and false negatives are called "type II error". Based on these numbers, several evaluations were created to give a better understanding of the results of the classification model [20].

Accuracy

The accuracy of a classification model is the ratio between one class's correct predictions to its total quantity. Mathematically it can be expressed in the following way:

$$\text{Accuracy} = (TP + TN) / (TP + FP + FN + TN)$$

Using accuracy can be misleading when used in a case where there is a high imbalance between the classes. Because if we just classify everything from the larger class we will get a big number of correct predictions and therefore a high accuracy score, but a misleading result of how well would the model function in the real world [20].

Precision

Precision measures the degree to which we receive the same outcomes from repeating the same experiments with the same conditions. Precision is also called the *positive prediction value*.

Informally both precision and accuracy are used interchangeably although they have different definitions, we can express precision mathematically in the following manner:

$$\text{Precision} = TP / (TP + FP)$$

The results of a model can be accurate but not precise, precise but not accurate, none of them or both of them at the same time, and a measurement is considered valid if it is both accurate and precise [20].

Recall

It is called sensitivity or the *true positive rate.* It is defined as the number of samples positively evaluated over the total number of the positive samples or mathematically [20]:

$$\text{Recall} = TP / (TP + FN)$$

F1

Known as F-score or F-measure as well. It is considered when making predictions in binary classification problems to check the accuracy of the model. It is expressed as the harmonic mean of both precision and recall, in the following way:

$$F1 = 2TP / (2TP + FP + FN)$$

The value of the F1-score is in the range [0, 1] and the closer it is to 1 the better, and it is usually used to interpret the general performance of the model [20].

KAPPA

Cohen's Kappa score measures the correspondence between the machine learning classifier and the observed classes and it is expressed mathematically in the following way:

$$KP = (accuracy - Pr(e))(1 - Pr(e))$$

The Kappa score takes values between -1 and 1 where 1 is the perfect score.

There are no values that are agreed upon on which we would say which scores are acceptable. The most common values are the ones mentioned in Figure 13 [30], [31].

Value of Kappa	Landis and Koch	Altman
<0.00	Poor	—
0.00–0.20	Slight	Poor
0.21–0.40	Fair	Fair
0.41–0.60	Moderate	Moderate
0.61–0.80	Substantial	Good
0.81–1.00	Almost perfect	Very good

Figure 13 Interpretation of Kappa score adapted from [30].

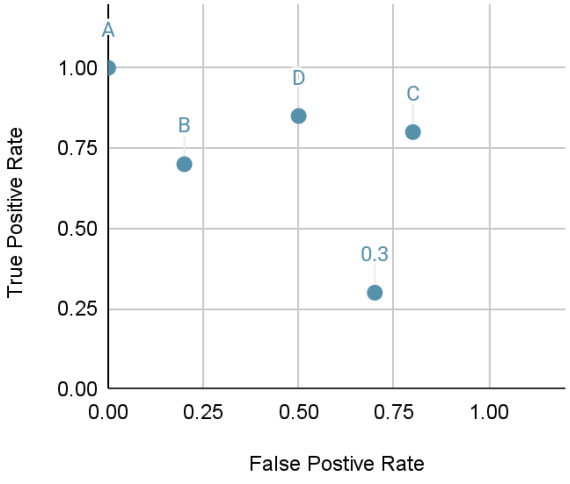
Receiver operating characteristics (ROC)

This technique for visualizing and organizing classifiers has been long used in the fields such as signal detection theory and in medicine as well for diagnostic systems performance. ROC graphs started gaining more and more popularity in machine learning as people realized that accuris not always the perfect measurement for the performance of machine learning models.

The graph of the ROC or the ROC space can help us understand the quality of a classifier and are usually presented as two inversely related variables are shown as in Figure X2 where the y-axis represents the True Positive Rate (TPR) and the x-axis False Positive Rate (FPR), where

$$FPR = \frac{FP}{P} \; and \; TPR = \frac{TP}{P}$$

Figure 14 ROC graph showing five discret classifiers



If we look at the graph on the right we can see that points like D and C are less strict as these classifiers have higher FPR, while the perfect case is point A which has a zero FPR. The metric that we obtain from the ROC curve is Area Under the ROC Curve (AUC) and it is the ratio of the unit square under the ROC curve, so the values of AUC are between [0,1], and no fairly good classifier should have AUC less than 0.5 [31], [32].

Federated Learning

FL is a machine learning paradigm in which many clients collectively train a machine learning model with the mentorship of a central server, keeping the training data on edge devices (the clients) [33].

The traditional paradigm of machine learning faces two main limitations. First, the privacy and safety concerns because of transferring the data of the users to central service providers. Second, the global model falls short when it comes to personalized patterns of each user or client. FL fixes the boundaries of traditional machine learning by having one global model and several local models [34].

Criteria	Centralized Learning	Federated Learning
Target	Collect data	Distribute learning
Training	Training on server	Training on edge devices
Aggregation	No aggregation	Aggregation on server
Model	Shared model	Personalized/Shared models
Sharing process	Data sharing	Model updates sharing
Iterations	One-time submission	Iterclient'socess

Figure 15 Comparison between centralized and federated learning approaches from [34]

One iteration of FL can be expressed in the following way [34]:

1. A number of the clients that will be involved in the FL gets chosen by a selection process.

2. All of the chosen clients receive the global model.

3. Every client trains the local model and uses the locally available data.

4. After receiving the local updates, the central server computes an aggregated model update. The clients that are struggling might be dropped to an efficient workflow, after having enough of the clients already sending back results. The process of aggregation could have several steps to enhance the security of the clients' data.

5. The server updates the global model depending on the aggregated model update received from the clients that were part of this round of training.

Federated Learning Properties and Suppositions

Federated Learning problems and systems have a common set of properties and suppositions that we will explain here

Non-IID Data

In traditional machine learning settings the training data is often presumed to be independent and identically distributed (IID), and this training data is usually collected and stored in a single storage [34].

According to Qinbin Li et al [35], data with non-iid distribution is a common challenge in the development of functional federated learning algorithms and in their research they work on 6 different ways in which it is possible to partition the data to achieve non–iid setting.

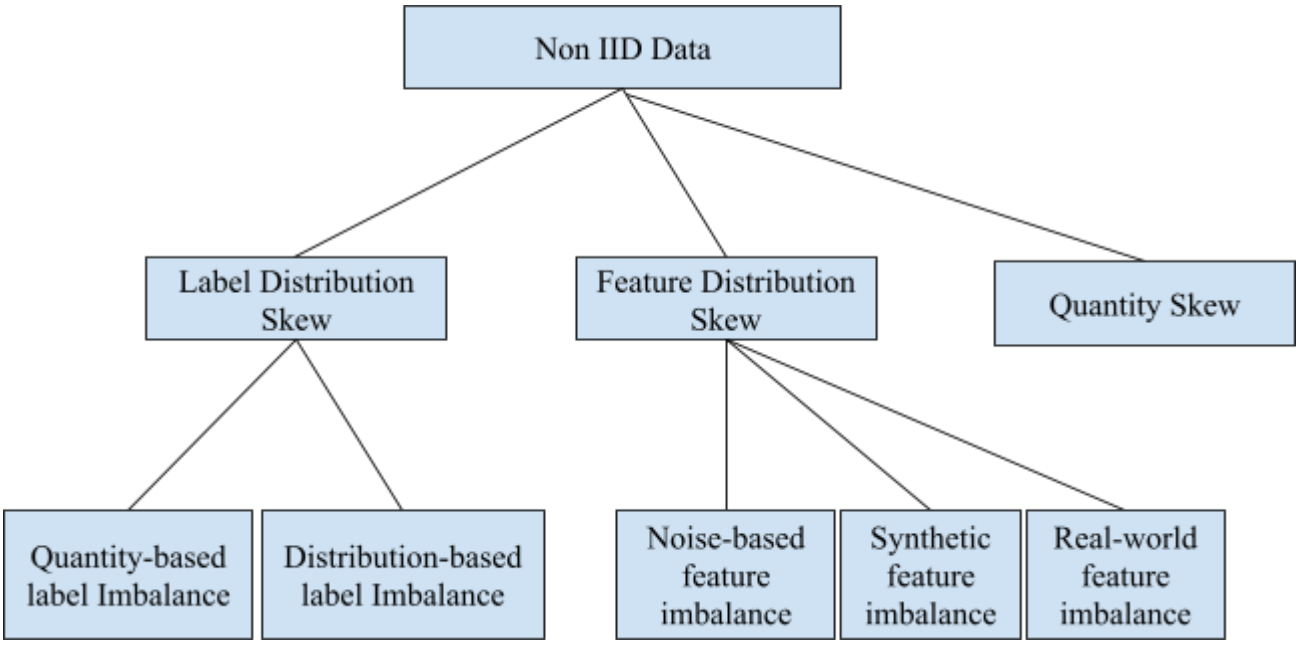


Figure 16 shows the different ways in which a dataset can be partitioned to get non-iid distribution

Label Distribution Skew

First of all, the label is the output of the machine learning model, and in this thesis this will be the class of the data sample. This is a common real-world situation where the clients have skew label distributions, for instance, a number of hospitals can be specialized in some kinds of diseases and as a result have more data samples from that population [35].

Quantity-based label imbalance

In this situation each client has a certain number of the labels [35]. This was first used with the FedAvg algorithm which we will discuss in a later chapter and the MNIST dataset [7] as the data is divided into subsets where each subset only has samples with the same labels and one subset was assigned to each client.

Distribution-based label imbalance

We will be using this method in the experiments as it is a good choice to represent real world data distributions, and it basically uses Dirichlet distribution to give a part of a certain label to each client. [35].

Feature Distribution Skew

The feature is the input of the model or the row data, and feature imbalance is when the distribution of the feature varies across the clients or the parties. It can be seen in a variety of scenarios. For example, the color of the coat of the cat can differ in pattern and color from one area to another.

Noise-based feature imbalance

In this scenario the dataset is divided into random equal parts that are assigned to the clients and then to each of the clients a different amount of Gaussian noise is added to the assigned part of the dataset [35].

Synthetic feature imbalance

This method is used to create an artificial feature imbalance and it can be done several ways one of them is mentioned in [35] where they create a federated dataset by assuming that the data lies in a cube and then use the axes of coordination xx`, yy`, and zz` which is in the middle of the cube to divide the cube into subsets, and assign the symmetrically opposite parts to the same client. Figure 15 can help visualize the idea.

Real-world feature imbalance

A good example about this is the relation between aging and cancer which is an acknowledged relation now, that with getting older it risk of getting cancer is higher [36]. This might create an imbalance in the age of the people in a dataset for cancer types for example.

Figure 17 (a) *Figure 17 (b)*

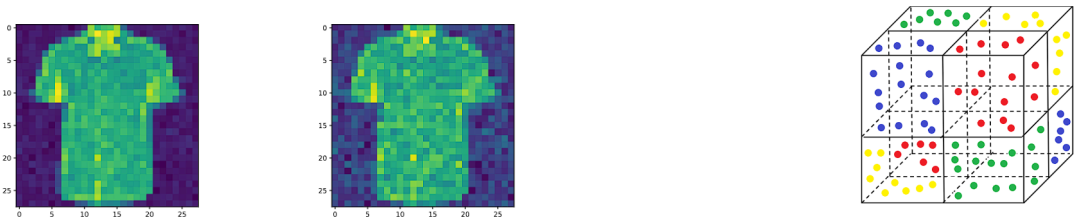


Figure 17 (a) shows the result of adding noise to a sample of the FMNIST dataset from a Gaussian distribution with different intensities, (b) a visualization of the dataset with synthetic feature imbalance called FCUBE adapted from [35].

Quantity Skew

When the data is quantity skewed, it means that there is no imbalance in the features or labels but rather in the quantity of the data between the clients. Assuming x is an amount of data that has a_1, a_2, …, a_n then the first client might have x data while the second 2x and consecutively 2a_1, 2a_2, …, 2a_n [35].

Optimization algorithms

FL training depends on local and global iterations trying to reach an efficient global model and handles data that is scattered on a set of clients. If we use the same techniques as traditional machine learning we would fall for short as they assume the possibility of accessing all the data [34].

Security and Privacy

The main purpose of FL was to make use of the data on the client devices without the need to access it which limits the risks of privacy and security [7].

Heterogeneous Devices

Not all of the clients in the real-world will have the same accessibility and computing power, which might affect the training time of the model [37].

Categories of Federated Learning

We can divide the architectures of Federated Learning into three categories based on the way the data is distributed: Horizontal Federated Learning, Vertical Federated Learning, and Federated Transfer Learning (FTL).

We will denote the data that client i holds by the matrix D_i, where each row of it is a sample of the data and each column is a feature. The feature space is denoted by X, the label space Y, and I to denote the sample **D** space [38].

Horizontal Federated Learning:

Also known as sample-based Federated Learning. It is used in the case where the entities have the same feature space but different samples, an example would be two banks that work in different regions, although the shared group of users is very small, the type of business is very similar and their datasets share very similar or similar feature spaces [38]. According to [38] we can outline horizontal federated learning in the following way

$$X_i = X_i, Y_i = Y_i, I_i \neq I_j, \forall D_i, D_j, i \neq j$$

Vertical Federated Learning:

Also known as feature-based federated learning is used when the clients have data with the same sample D space but their feature spaces are different. An example could be a bank and a chain store in the same city. While sharing almost the same set of customers, the feature spaces can be entirely different as the chain store would keep track of the customers' purchasing history and their preferences and the bank would keep track of customers' revenue and credit rating [38].

In order to create a model from the data of every client, the features are aggregated and then the training loss and gradients are computed in a privacy-preserving way. It can be outlined in the following way [38]:

$$X_i \neq X_j, Y_i \neq Y_j, I_i = I_j \forall D_i, D_j, i \neq j$$

Federated Transfer Learning (FTL):

This happens in cases where the clients don't have a lot in common as they differ in both the feature spaces and the sample spaces. The situation where we have a financial institution located in China and a chain store in the United States. The set of clients of the two organizations would have a little

to no intersection because of the distance factor. Also the set of the features would share a small portion as the type of data that interests them is different.

This technique is important as it handles problems that are beyond the current scope of federated learning algorithms, and it can be outlined in the following way:

$$X_i \neq X_j, Y_i \neq Y_j, I_i \neq I_j \forall D_i, Dj, i \neq j$$

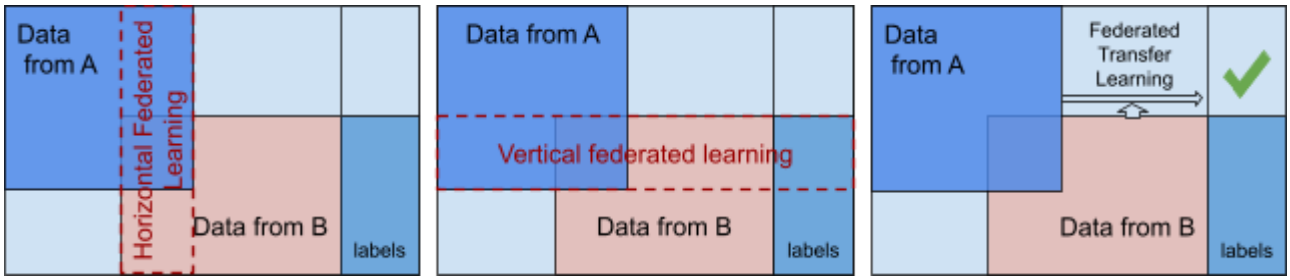


Figure 18 Different categories of Federated Learning adapted from [38]

Electroencephalography (EEG)

To put it in simple words an EEG is a test that records the brain activity using electrodes that are sticked on the scalp of the head, the electrodes can be described as metal discs with thin wires. EEG tests are used in the identification or treatment of brain disorders and illnesses like epilepsy, and as we will see in the next section they can be used for emotion classification [39].

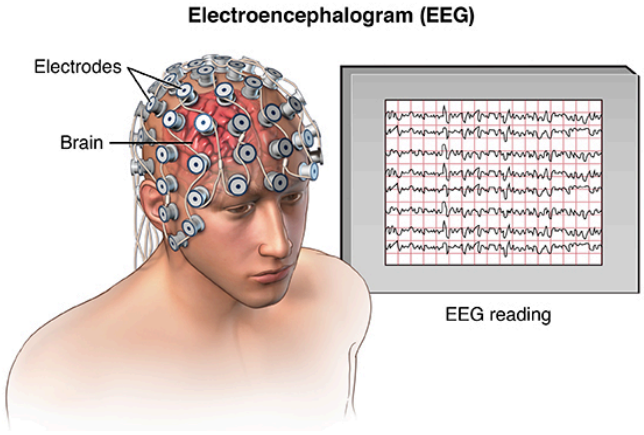


Figure 19 The EEG measurement and display using electrodes on the brain [40]

EEG data have the following traits [41], [42]:

1- EEG recordings are noisy and sensitive to interference from the environment. Typically, the EEG

signals are combined with noise, interference, artifacts, and several additional signals (such the EOG, EMG, and ECG).

2- The data coming from EEG recordings can be divided into two classes, spontaneous or evoked. During the recording process, different outer physiological signals can affect the signals that are being recorded.

3- The EEG signal's fluctuation is highly non-stationary, unstable, and prone to external environmental variables. Statistical analysis methods are widely used in research to find and classify EEG signal characteristics.

4- The part of EEG signals that is most related to cognitive activities is the one with frequency between 0.5 – 30 Hz, which is a part of the frequency range of EEG signals which is 0.5 – 100 Hz.

3.Related Work

Federated learning algorithms, though they are relatively new, have already been used in a variety of fields [43], [44], [45] and the most common applications now are in the medical field, smart cities and finance. On a more specific aspect, Federated Learning algorithms have been used for EEG classification with different architectures and models [46], [47], [48].

Federated Learning in Healthcare

Machine learning algorithms get better and more accurate with more data, and federated learning, by giving the chance for multiple parties to train hand in hand machine learning models without the necessity to share data, gives us the chance to create stronger and less biased models. These are the main reasons why federated learning is gaining more and more attraction in the medical field where the sensitivity of the data plays a major role. Nowadays the applications of federated learning already include medical image analysis [49], disease prediction [50] , and patient monitoring [51].

Classification of EEG Data

Several machine learning techniques have been used for EEG data classification, for example in [52] convolutional neural networks were used, and in [53] uses a mix of convolutional neural networks and LSTMs for the classification of intracerebral EEG data. We can see as [54] used transformer based models showed good results for the classification of EEG data during motor imagery tasks.

Federated Learning for EEG data classification

Xu et al [47] have explored in their work the usage of federated learning algorithms for emotion recognition using EEG data. In their work they first extracted the features which are properties of the signals that are identified in a portion of the pattern [55] and they form an important factor keeping the valuable information from being lost.
The model they used consists of a Convolutional Neural Network (CNN) which they trained on the extracted features and the federated learning algorithm they used was a FedAvg algorithm which we will explain in detail e related applications. However, non-IID data scenarios and more complex federated learning in the next section. When comparing the results between IID data distribution and

non-iid, they found out that the model trained on IID data was 14.89% more accurate than the one trained on Non-IID data.

The current research, as we can see, shows the effectiveness of federated learning approaches in the task of EEG data classification and other healthcare techniques on EEG remain an unexplored area which this thesis aims to cover.

4.Methods

The Datasets and Classes

The data that is used in this thesis is intracerebral EEG (iEEG) data from [52] which was collected from two institutions. First, the Department of Neurology, St. Anne's University(Brno, Czech Republic) and Mayo Clinic (Rochester, Minnesota, U.S.A). The data from St.Anne's University Hospital was obtained from iEEG recordings from 11 patients of drug resistant epilepsy (DRE). These patients were going under an evaluation for epilepsy surgery. The patients were implanted with standard intracranial depth electrodes (semi-flexible multi-contact platinum electrodes with varying contacts: 5, 10, and 15). These electrodes had a diameter of 0.8 mm, a contact length of 2 mm, a contact surface area of 5.02 mm^2, and an inter-contact distance of 1.5 mm. A 192-channel research EEG acquisition system (M&I; Brainscope, Czech Republic) was utilized to record 30 minutes of awake resting interictal intracranial EEG (iEEG) data at a sampling rate of 25 kHz with a common reference montage. The raw data was filtered to a bandwidth of 2 kHz and downsampled to 5 kHz. The study adhered to ethical standards, with approval from the St. Anne's University Hospital Research Ethics Committee and the Ethics Committee of Masaryk University. Written informed consent was obtained from all subjects in accordance with the Declaration of Helsinki.

The data from Mayo Clinic was made up of iEEG recordings collected from 25 DRE patients who were going under an evaluation for epilepsy surgery. Data was collected using a Neuralynx Cheetah system as segments of 2 hours long on the first night after the patient's arrival at the intensive care unit (ICU) between 1 and 3 AM and at a sampling rate of 32 kHz with a hardware filter bandwidth of DC - 9 kHz. After that, an antialiasing filter was applied using Bartlett-Hanning window at 1 kHz and then the data was down-sampled to 5 kHz. Patients were implanted with depth electrodes made of 4 or 8 Platinum/Iridium contacts which were 2.3 mm in length and 1 mm in diameter, spaced 5 or 10 mm apart. The study was in correspondence with the guidelines provided by the Mayo Clinic Institutional Review Board, as all of the subjects provided a written consent following the principles of the Declaration of Helsinki.

The data from both institutions was manually annotated, and was divided into four classes which are (Physiological iEEG, Pathological iEEG, Non-cerebral artifacts, Line noise). The Line noise class has all data segments with line noise 50 or 60 Hz, the segments that come from muscles, movements or machine artifacts are in the Non-cerebral artifacts class, segments that has pathological activities

like interictal spikes are in the Pathological iEEG class, and the remaining segments are in the physiological iEEG class.

Data preprocessing

The data preprocessing included the creation of a custom Pytorch class which is based on the work of Petr Nejedly [52] with the following note:

the `__getitem__` loads the data from the `MATLAB` files using `Scipy` library loadmat function and `mat73` which is a custom library which allows to load `MATLAB 7.3` files with a try except as not all the files have the same version of `MATLAB`. After that, we use the `spectrogram` function from the `Scipy` library to get the power spectrum values at each frequency and time bin, finally the data gets normalized and returns the data and target which is the label of the iEEG signals.

Training

The dataset is collected from two institutes and is non-IID, and is skewed towards the class Physiological iEEG, having the majority of samples, this is why we use downsampling. We are using 13784 samples from each class which corresponds to the lowest number of samples in these three classes.

For the distribution of the data, there are three scenarios being considered. The first one is when the data has IID distribution and each client has the same number of total samples and the same number of samples from each class. In the second one we use Dirichlet distribution to create a distribution-based label imbalance. The last one is when we have one or two classes only available for each one of the clients but not all of the three classes.

The code has be written and excecuted on Kaggle notebooks and the models where built using Pytorch.

Model

The model that is used in this study is based on a modified version of the model used in [53] implemented by professor Petr Nejedly which is a CNN-GRU [56], as it provides more comprehensible results, and maintains the basic advantages of LSTM with a shorter training time.

The model was built using Pytorch deep-learning framework. The convolutional layer which is for

the extraction of features from spectrograms contains 256 filters with kernel size 200×3, which was chained with a batch normalization layer and Rectified Linear Unit (ReLU) layer. The resulting vectors were passed to a Gated Recurrent Unit (GRU) with input size 256 and output size 128. After that a drop layer with probability 0.5 and the last layer is a fully connected layer with input size 128 and 3 as an output size which corresponds to the number of classes.

Federated Learning Algorithms

The federated learning algorithms used in the training of the model are FedAvg and SCAFFOLD. FedAvg is an algorithm first suggested by McMahan et al [7] in 2016 so that we can make use of the huge amounts of data we generate and without exposing it or risk its privacy. The two main ideas of FedAvg are (1) each of the clients locally updates the model. (2) the server receives and aggregates clients' updates and uses them to update the global model parameters.

In the following block is the pseudocode for the algorithm as it is mentioned in [7].

Algorithm 1 FedAvg. The K clients are indexed by k; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

initialize w_0

for each round t = 1, 2, ... do

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$ (random set of m clients)

for each client $k \in S_t$ in parallel do

$w^k_{t+1} \leftarrow \text{ClientUpdate}(k, w_t)$

$m_t \leftarrow \sum_{k \in St} n_k$

$w^k_{t+1} \leftarrow \sum_{k \in St} (n_k \div m_t) w^k_{t+1}$

ClientUpdate(k, w): // Run on client k

$B \leftarrow$ (split P_k into batches of size B)

for each local epoch i from 1 to E do

for batch $b \in B$ do

$w \leftarrow w - \eta \nabla \ell(w; b)$ return w to server

The second algorithm which we are using is SCAFFOLD [57] which works on fixing the "drift" that FedAvg experiences in case of heterogeneous data. As if we have two clients, client 1 and client 2 where the data each one of them has is different from the data the other has, then the local gradient will move to x 1 and x 2 where x 1 and x 2 are the optimums for client 1 and client 2 respectively and they are away from each other. The average between client 1 and client 2 won't correspond to the real global optimum for these two clients as they have different data distributions.

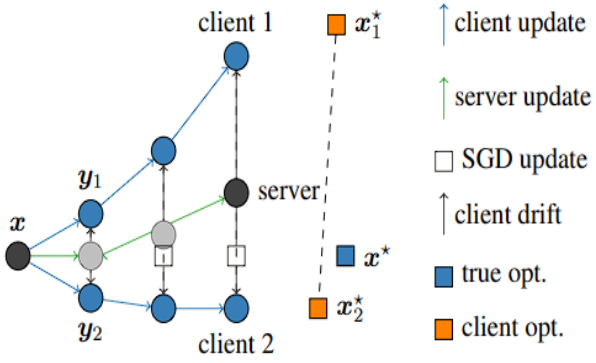


Figure 20 explaining the "drift" that is caused by the heterogeneity of the clients' data.

Before we delve deeper into the details, we will use the following notation which is the same as the one used in the original paper:

N (total number of clients), S (samples number of the clients), i (index of the clients)

R (number of the communication rounds), r (index of the communication round).

K (number of local update steps), k (index of the local update steps)

x^r (aggregated server model after round r)

$y_{i,k}^r$ (i th client's model in round r and step k)

c^r (control variate of the server), c_i^r (i th client after round r)

Scaffold algorithm also can be explained in three main steps. (1) the local update of clients' parameters or in other words the local model of each client. (2) update the control variates of each client locally. (3) aggregating the updates.

The key difference in SCAFFOLD is the introduction of control variates both for the clients and the server and in the paper they are noted as c_i and c respectively. The way we initialize these variates guarantees that $c = 1/N \sum c_i$ and it is possible to initialize them to zero safely. Every round of

training the central model parameters (x, c) are sent to the clients involved in this training round, and every one of these clients sets the local model with the global model, after which it will perform a K local passes on the data that it has in the following form:

$$y_i \leftarrow y_i - \eta_l(g_i(y_i) + c - c_i$$

After that, the local control variate c_i is updated as well. There are two choices for updating the local control variate:

$$(1) \qquad c_i^+ = g_i(x)$$

$$(2) \qquad c_i - c + \frac{1}{K\eta_l}(x - y_i)$$

The second option is usually the option that is opted to as it costs less in terms of computation, because it uses the previously computed gradients for updating the control variate. On the other hand, the first option will go one more time over the local data to compute the gradient at the server model x. One advantage of the first option is its stability but as it needs one more round of gradient computation it is more expensive than the second choice which is ,though less stable, good enough. The last step in which one round of communication ends is the aggregation of the clients' updates and then used to update the parameters of the server:

$$x \leftarrow x + \frac{\eta_g}{|S|}\sum_{i\in S}(y_i - x)$$

$$c \leftarrow c + \frac{1}{N}\sum_{i\in S}(c_i^+ - c_i)$$

An important note is that if c_i is always 0 then the algorithm becomes analogous to FedAvg.

Algorithm 2 SCAFFOLD. Stochastic Controlled Averaging for federated learning

---

Server input: initial x and c, and global step-size η_g
client i's input: c_i, and local step-size η_l
for each round r = 1, . . . , R do
    sample clients S ⊆ {1, . . . , N}
    communicate (x, c) to all clients i ∈
    $S_t \leftarrow$ (random set of m clients)
    on client i ∈ S **in parallel do**

```
        initialize local model yi ← x
        for k = 1, . . . , K do
              compute mini-batch gradient gi(yi)
              yi ← yi − ηl (gi(yi) − ci + c)
        end for
        c + i ← (i) gi(x), or (ii) ci − c + 1 Kηl (x − yi)
        communicate (Δyi , Δci) ← (yi − x, c + i − ci)
        ci ← c + i
    end on client
    (Δx, Δc) ← 1 |S| P i∈S (Δyi , Δci)
    x ← x + ηgΔx and c ← c + (|S|/N) Δc
end for
```

5.Results

The results are compared with a baseline implemented with a centralized model which used 13784 samples from each of the classes except the line noise class. The results that were achieved in this case are the following: **F1**: **0.78**, **KAPPA**: **0.71, and AUROC**: **0.94**

In this thesis, we checked the accuracy of federated learning algorithms for the classification of a multiclass dataset of intracerebral EEG data collected from two different institutes which mimics the real-world environment of federated learning. In Figure 24 we show the different Kappa scores achieved by the Scaffold and Fedavg on the IID setting, Label Distribution Skew (Weak)

Non-IID setting, and Quantity-based Label Imbalance (Extreme) Non-IID setting. Figures 21 and 22 clarify how the data is distributed across the clients in both cases. Note that the Weak Non-IID setting depends on Dirichlet distribution for 40 clients, and the IID is created by dividing each class on each one of the 40 clients equally.

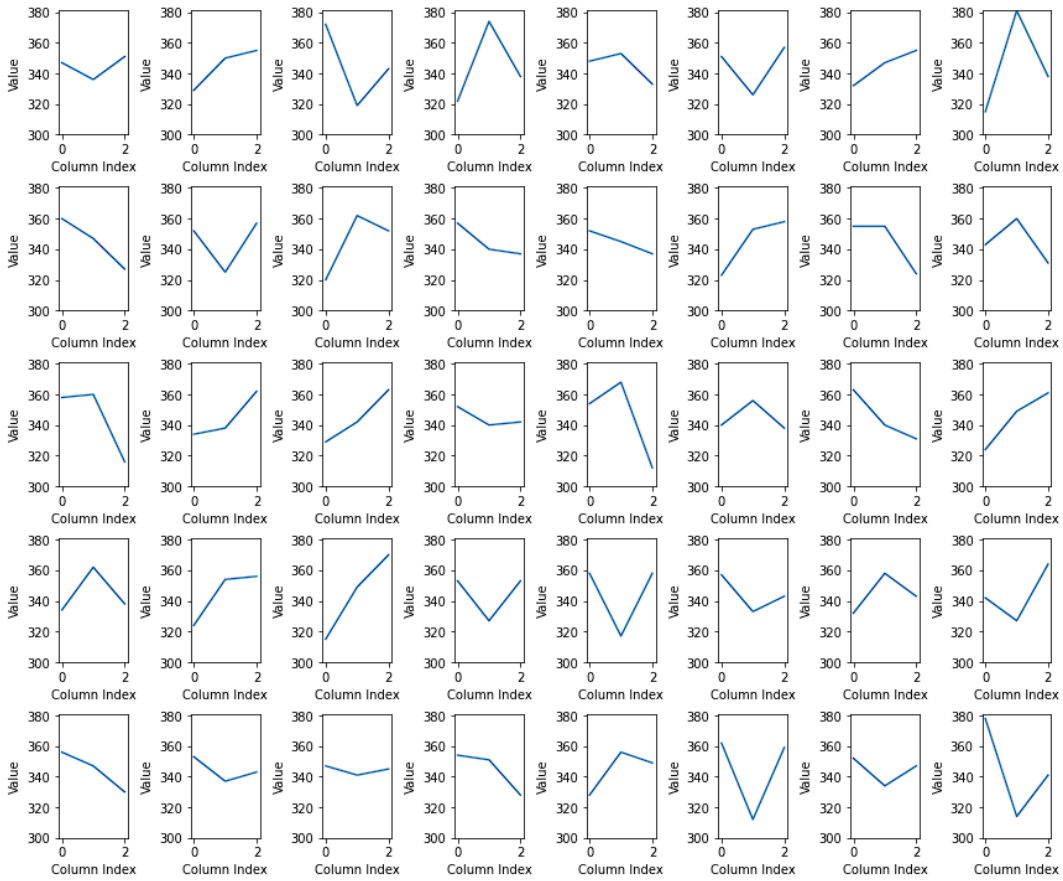


Figure 21 shows the distribution of the data on different clients using Dirichlet distribution to assign part of each class to each of the clients.

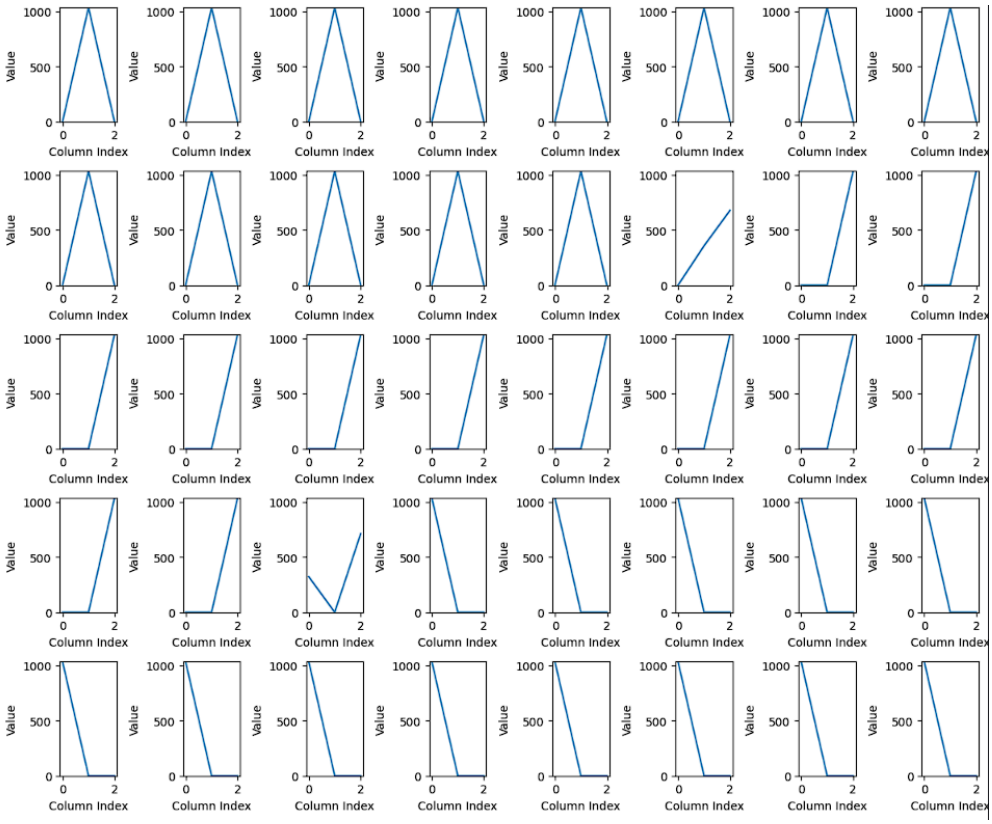


Figure 22 shows the distribution of the data on different clients where each client has only samples of one or two classes out of the three classes.

First, we used the FedAvg algorithm as it is a more simple algorithm, and we compared the results we achieved on the IID, the Weak Non-IID, and extreme Non-IID. Figure 23 shows the confusion matrix of the results achieved on the test dataset (Mayo Clinic) of the FedAvg model after running it for 15 rounds and a learning rate of 0.001.

			PREDICTED					
EXPICTED	FedAvg		Noise and muscle activity		Physiological iEEG		Pathological iEEG	
	Noise and muscle activity	IID	23922	22437	1090	2261	375	690
		Weak Non-IID	22697		2492		198	
		Extreme Non-IID	***4***		24779		606	
	Physiological iEEG	IID	7218	3341	31420	35389	5620	5527
		Weak Non-IID	5283		34482		4493	
		Extreme Non-IID	1		***43450***		805	
	Pathological iEEG	IID	65	58	390	618	5644	5423
		Weak Non-IID	55		745		5299	
		Extreme Non-IID	0		2785		***3314***	

Figure 23 the confusion matrix of FedAvg on different settings of data distribution with respect to the centralized baseline.

Finally we compare the performances of FedAvg and Scaffold on each of the settings of data distribution. We notice easily that the performance of FedAvg on the test dataset which is Non-IID can be inferred from the accuracies in the validation in the cases of IID and Weak Non-IID, but in the case of Extreme Non-IID, had unpredictable behavior which oscillates between two accuracy with big difference in accuracy even after ***100*** training rounds Figure 24 clarifies this behavior.

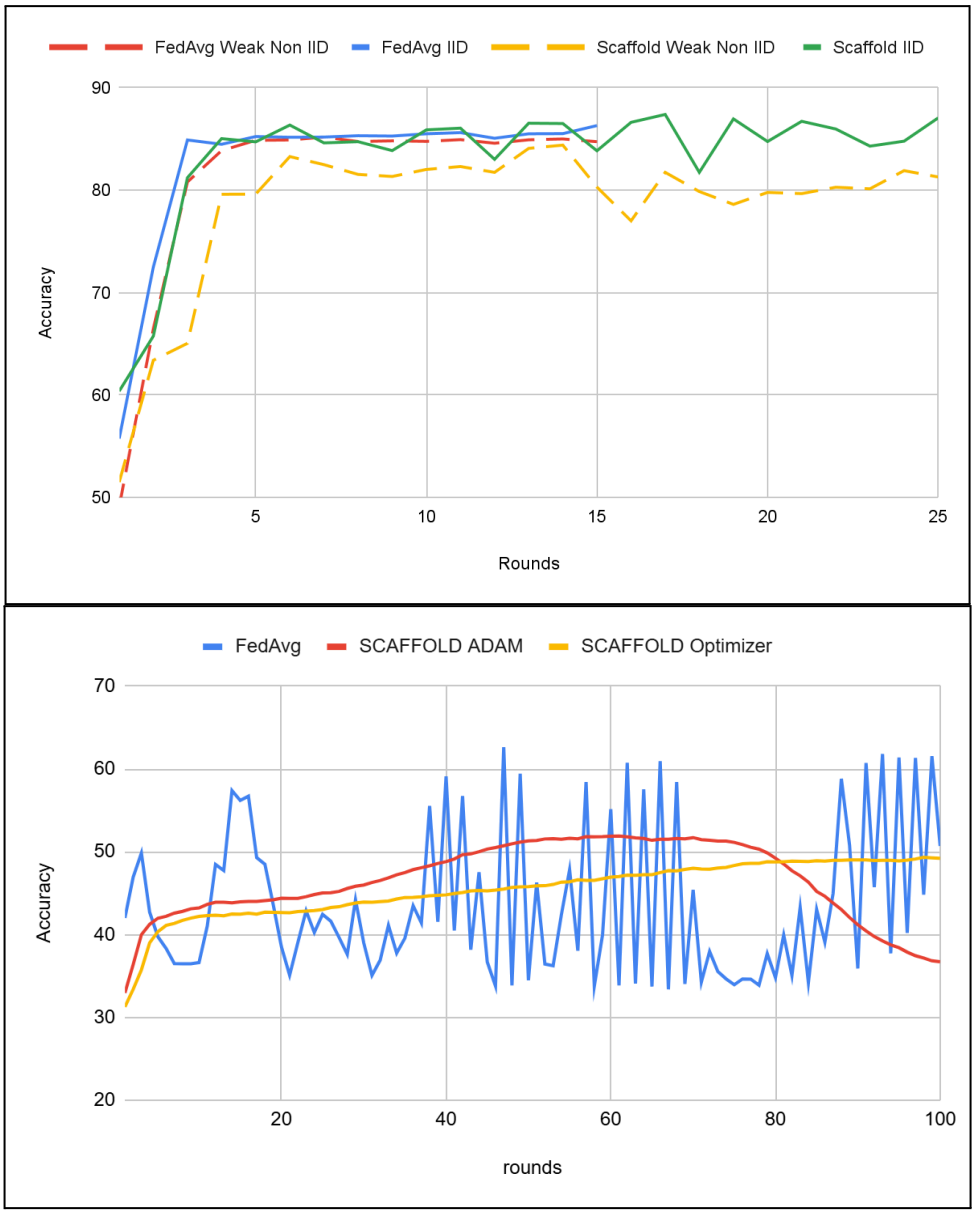


Figure 24 (up) shows the accuracies of FedAvg and SCAFFOLD on the cases of IID and Weak Non-IID data distribution. (Down) shows the accuracies for FedAvg and SCAFFOLD on Extreme Non-IID data distribution case.

	FedAvg			Scaffold			
	ADAM			ADAM			Scaffold optimizer
Data Setting	IID	Weak Non IID	Extreme Non IID	IID	Weak Non IID	Extreme Non IID	Extreme Non IID
Learning Rate	0.001	0.001	0.001	0.001	0.001	0.0003	0.0001
Number of Rounds	15	15	100	25	25	100	100
F1 Score	0.76	0.78	0.45	0.66	0.7	0.31	0.5
Kappa Score	0.67	0.7	0.14	0.53	0.57	0.05	0.3
AUROC	0.94	0.94	0.57	0.94	0.93	0.7	0.69

Figure 25 shows the different training configurations and results for FedAvg and SCAFFOLD.

Neither FedAvg nor SCAFFOLD with ADAM optimizer performed well in the case of Extreme Non-IID data distribution, this is why we tried one more optimizer for SCAFFOLD which inherits from PyTorch's optimizer class, Scaffold Optimizer updates model parameters during training. It performs a standard gradient descent step with the learning rate, but also incorporates server-side and client-side controls into the update direction. This optimizer achieves about 10% percent more accuracy than FedAvg and 20% more than the case when ADAM is used with SCAFFOLD. Also, as shown in Figure 24 it is the only one that is steady in the training but unfortunately due to lack of computational capabilities, it can’t be further explored on more rounds of training.

6. Conclusions

The importance of the privacy of the users' data and the lack of data that is sufficient to train machine learning models, and as a response, federated learning algorithms are a popular solution. In this thesis, we have introduced and analyzed two of the best federated learning algorithms for iEEG data classification. The significance of this study is that we used a separate dataset for testing the performance of the models. Moreover, we have also highlighted the difference in performance between FedAvg and Scaffold on different data settings and different learning rates. With FedAvg we were able to achieve results very close to the centralized baseline with 0.7 Kappa Score compared to 0.71 achieved in the centralized baseline. Using SCAFFOLD it was not possible to achieve the same resutls the baseline. Working on mitigating the effect the non-homogeneity of the data can create, we used SCAFFOLD with a small learning rate which was able to achieve a 0.3 Kappa score which is double the one achieved by FedAvg on the Extreme-Non-IID case.

One of the main limitations of this study was computational power as the only available option was Kaggle, trying to run the model on the Extreme-Non-IID with a small learning rate and more than 100 iterations could be investigated further.

Bibliography

[1] H. Sheikh, C. Prins, and E. Schrijvers, "Artificial Intelligence: Definition and Background," in *Mission AI*, in Research for Policy. , Cham: Springer International Publishing, 2023, pp. 15–41. doi: 10.1007/978-3-031-21448-6_2.

[2] M. B. and Ma. Louella, "Bitter Melon Crop Yield Prediction using Machine Learning Algorithm," *Int. J. Adv. Comput. Sci. Appl.*, vol. 9, no. 3, 2018, doi: 10.14569/IJACSA.2018.090301.

[3] J. A. Nichols, H. W. Herbert Chan, and M. A. B. Baker, "Machine learning: applications of artificial intelligence to imaging and diagnosis," *Biophys. Rev.*, vol. 11, no. 1, pp. 111–118, Feb. 2019, doi: 10.1007/s12551-018-0449-9.

[4] "How Many Photos Are There? (Statistics & Trends in 2024)." Accessed: Mar. 14, 2024. [Online]. Available: https://photutorial.com/photos-statistics/

[5] S. Tufail, H. Riggs, M. Tariq, and A. I. Sarwat, "Advancements and Challenges in Machine Learning: A Comprehensive Review of Models, Libraries, Applications, and Algorithms," *Electronics*, vol. 12, no. 8, Art. no. 8, Jan. 2023, doi: 10.3390/electronics12081789.

[6] N. Truong, K. Sun, S. Wang, F. Guitton, and Y. Guo, "Privacy preservation in federated learning: An insightful survey from the GDPR perspective," *Comput. Secur.*, vol. 110, p. 102402, Nov. 2021, doi: 10.1016/j.cose.2021.102402.

[7] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," 2016, doi: 10.48550/ARXIV.1602.05629.

[8] I. H. Sarker, "Machine Learning: Algorithms, Real-World Applications and Research Directions," *SN Comput. Sci.*, vol. 2, no. 3, p. 160, May 2021, doi: 10.1007/s42979-021-00592-x.

[9] J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 2022.

[10] J. Han and M. Kamber, *Data mining: concepts and techniques*, 2nd ed. in The Morgan Kaufmann series in data management systems. Amsterdam ; Boston : San Francisco, CA: Elsevier ; Morgan Kaufmann, 2006.

[11] W. Xing and Y. Bei, "Medical Health Big Data Classification Based on KNN Classification Algorithm," *IEEE Access*, vol. 8, pp. 28808–28819, 2020, doi: 10.1109/ACCESS.2019.2955754.

[12] M. Gor *et al.*, "Density Prediction in Powder Bed Fusion Additive Manufacturing: Machine Learning-Based Techniques," *Appl. Sci.*, vol. 12, no. 14, p. 7271, Jul. 2022, doi: 10.3390/app12147271.

[13] G. H. John and P. Langley, "Estimating Continuous Distributions in Bayesian Classifiers," 2013, doi: 10.48550/ARXIV.1302.4964.

[14] G. I. Webb, "Naïve Bayes," in *Encyclopedia of Machine Learning*, C. Sammut and G. I. Webb, Eds., Boston, MA: Springer US, 2011, pp. 713–714. doi: 10.1007/978-0-387-30164-8_576.

[15] C. Manning, P. Raghavan, and H. Schuetze, "Introduction to Information Retrieval," 2009.

[16] E. Bisong, "Logistic Regression," in *Building Machine Learning and Deep Learning Models on Google Cloud Platform*, Berkeley, CA: Apress, 2019, pp. 243–250. doi: 10.1007/978-1-4842-4470-8_20.

[17] S. S. Haykin, *Neural networks and learning machines*, 3. ed. New York Munich: Prentice-Hall, 2009.

[18] *Multivariate Statistical Machine Learning Methods for Genomic Prediction*. Accessed: Mar. 31, 2024. [Online]. Available: https://link.springer.com/book/10.1007/978-3-030-89010-0
[19] F. C. Bravo and J. A. L. Sotelo, *Una aproximación práctica a las redes neuronales artificiales*, 1st ed. Universidad del Valle, 2009. Accessed: Apr. 01, 2024. [Online]. Available: https://www.jstor.org/stable/j.ctv1228hwf
[20] "Patterson, J. and Gibson, A. (2017) Deep Learning A Practitioner's Approach. O'Reilly Media, Inc., Sebastopol. - References - Scientific Research Publishing." Accessed: Apr. 01, 2024. [Online]. Available: https://www.scirp.org/reference/ReferencesPapers?ReferenceID=2283763
[21] "Backpropagation | Brilliant Math & Science Wiki." Accessed: Jun. 03, 2024. [Online]. Available: https://brilliant.org/wiki/backpropagation/#formal-definition
[22] "Feedforward Neural Networks | Brilliant Math & Science Wiki." Accessed: Jun. 03, 2024. [Online]. Available: https://brilliant.org/wiki/feedforward-neural-networks/#formal-definition
[23] "Introduction to Backpropagation Neural Network Computation | Pharmaceutical Research." Accessed: Jun. 03, 2024. [Online]. Available: https://link.springer.com/article/10.1023/A:1018966222807
[24] T. Szandała, *Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks*. 2020.
[25] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, "Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark." arXiv, Jun. 28, 2022. Accessed: Apr. 15, 2024. [Online]. Available: http://arxiv.org/abs/2109.14545
[26] "CS231n Convolutional Neural Networks for Visual Recognition." Accessed: Apr. 16, 2024. [Online]. Available: https://cs231n.github.io/neural-networks-1/
[27] C. Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall, "Activation Functions: Comparison of trends in Practice and Research for Deep Learning." arXiv, Nov. 08, 2018. Accessed: Apr. 16, 2024. [Online]. Available: http://arxiv.org/abs/1811.03378
[28] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." arXiv, Sep. 02, 2014. Accessed: Apr. 19, 2024. [Online]. Available: http://arxiv.org/abs/1406.1078
[29] N. Li, L. Hu, Z.-L. Deng, T. Su, and J.-W. Liu, "Research on GRU Neural Network Satellite Traffic Prediction Based on Transfer Learning," *Wirel. Pers. Commun.*, vol. 118, no. 1, pp. 815–827, May 2021, doi: 10.1007/s11277-020-08045-z.
[30] M. Bland, *An introduction to medical statistics*, Fourth edition. in Oxford medical publications. Oxford: Oxford University Press, 2015.
[31] I. M. De Diego, A. R. Redondo, R. R. Fernández, J. Navarro, and J. M. Moguerza, "General Performance Score for classification problems," *Appl. Intell.*, vol. 52, no. 10, pp. 12049–12063, Aug. 2022, doi: 10.1007/s10489-021-03041-7.
[32] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006, doi: 10.1016/j.patrec.2005.10.010.
[33] P. Kairouz *et al.*, "Advances and Open Problems in Federated Learning," *Found. Trends® Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, Jun. 2021, doi: 10.1561/2200000083.
[34] S. Banabilah, M. Aloqaily, E. Alsayed, N. Malik, and Y. Jararweh, "Federated learning review: Fundamentals, enabling technologies, and future applications," *Inf. Process. Manag.*, vol. 59, no. 6, p. 103061, Nov. 2022, doi: 10.1016/j.ipm.2022.103061.
[35] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated Learning on Non-IID Data Silos: An Experimental Study." arXiv, Oct. 28, 2021. doi: 10.48550/arXiv.2102.02079.

[36] L. Berben, G. Floris, H. Wildiers, and S. Hatse, "Cancer and Aging: Two Tightly Interconnected Biological Processes," *Cancers*, vol. 13, no. 6, p. 1400, Mar. 2021, doi: 10.3390/cancers13061400.

[37] Z. Chai *et al.*, "Towards Taming the Resource and Data Heterogeneity in Federated Learning," presented at the 2019 USENIX Conference on Operational Machine Learning (OpML 19), 2019, pp. 19–21. Accessed: Apr. 28, 2024. [Online]. Available: https://www.usenix.org/conference/opml19/presentation/chai

[38] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated Machine Learning: Concept and Applications." arXiv, Feb. 13, 2019. Accessed: May 03, 2024. [Online]. Available: http://arxiv.org/abs/1902.04885

[39] "Electroencephalogram (EEG)." Accessed: May 15, 2024. [Online]. Available: https://www.hopkinsmedicine.org/health/treatment-tests-and-therapies/electroencephalogram-eeg

[40] Y. Shen, "Machine Learning Based Epileptic Seizure Detection for Responsive Neurostimulator System Optimization," *J. Phys. Conf. Ser.*, vol. 1453, no. 1, p. 012089, Jan. 2020, doi: 10.1088/1742-6596/1453/1/012089.

[41] J. Zhang, Z. Yin, P. Chen, and S. Nichele, "Emotion recognition using multi-modal data and machine learning techniques: A tutorial and review," *Inf. Fusion*, vol. 59, pp. 103–126, Jul. 2020, doi: 10.1016/j.inffus.2020.01.011.

[42] E. H. Houssein, A. Hammad, and A. A. Ali, "Human emotion recognition from EEG-based brain–computer interface using machine learning: a comprehensive review," *Neural Comput. Appl.*, vol. 34, no. 15, pp. 12527–12557, Aug. 2022, doi: 10.1007/s00521-022-07292-4.

[43] S. Duan, D. Zhang, Y. Wang, L. Li, and Y. Zhang, "JointRec: A Deep-Learning-Based Joint Cloud Video Recommendation Framework for Mobile IoT," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 1655–1666, Mar. 2020, doi: 10.1109/JIOT.2019.2944889.

[44] I. Dayan *et al.*, "Federated learning for predicting clinical outcomes in patients with COVID-19," *Nat. Med.*, vol. 27, no. 10, pp. 1735–1743, Oct. 2021, doi: 10.1038/s41591-021-01506-3.

[45] J. C. Jiang, B. Kantarci, S. Oktug, and T. Soyata, "Federated Learning in Smart City Sensing: Challenges and Opportunities," *Sensors*, vol. 20, no. 21, p. 6230, Oct. 2020, doi: 10.3390/s20216230.

[46] C. Ju, D. Gao, R. Mane, B. Tan, Y. Liu, and C. Guan, "Federated Transfer Learning for EEG Signal Classification," in *2020 42nd Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, Jul. 2020, pp. 3040–3045. doi: 10.1109/EMBC44109.2020.9175344.

[47] C. Xu, H. Liu, and W. Qi, "EEG Emotion Recognition Based on Federated Learning Framework," *Electronics*, vol. 11, no. 20, Art. no. 20, Jan. 2022, doi: 10.3390/electronics11203316.

[48] M. Ghader, B. Farahani, Z. Rezvani, M. Shahsavari, and M. Fazlali, "Exploiting Federated Learning for EEG-based Brain-Computer Interface System," in *2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Berlin, Germany: IEEE, Jul. 2023, pp. 1–6. doi: 10.1109/COINS57856.2023.10189325.

[49] H. Guan, P.-T. Yap, A. Bozoki, and M. Liu, "Federated Learning for Medical Image Analysis: A Survey." arXiv, Sep. 12, 2023. doi: 10.48550/arXiv.2306.05980.

[50] S. Bebortta, S. S. Tripathy, S. Basheer, and C. L. Chowdhary, "FedEHR: A Federated Learning Approach towards the Prediction of Heart Diseases in IoT-Based Electronic Health

Records," *Diagnostics*, vol. 13, no. 20, p. 3166, Oct. 2023, doi: 10.3390/diagnostics13203166.
[51] A. Mishra, S. Saha, S. Mishra, and P. Bagade, "A federated learning approach for smart healthcare systems," *CSI Trans. ICT*, vol. 11, no. 1, pp. 39–44, Apr. 2023, doi: 10.1007/s40012-023-00382-1.
[52] P. Nejedly *et al.*, "Intracerebral EEG Artifact Identification Using Convolutional Neural Networks," *Neuroinformatics*, vol. 17, no. 2, pp. 225–234, Apr. 2019, doi: 10.1007/s12021-018-9397-6.
[53] P. Nejedly *et al.*, "Exploiting Graphoelements and Convolutional Neural Networks with Long Short Term Memory for Classification of the Human Electroencephalogram," *Sci. Rep.*, vol. 9, no. 1, p. 11383, Aug. 2019, doi: 10.1038/s41598-019-47854-6.
[54] J. Xie *et al.*, "A Transformer-Based Approach Combining Deep Learning Network and Spatial-Temporal Information for Raw EEG Classification," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 30, pp. 2126–2136, 2022, doi: 10.1109/TNSRE.2022.3194600.
[55] M. K. Delimayanti *et al.*, "Classification of Brainwaves for Sleep Stages by High-Dimensional FFT Features from EEG Signals," *Appl. Sci.*, vol. 10, no. 5, Art. no. 5, Jan. 2020, doi: 10.3390/app10051797.
[56] P. Nejedly, "xnejed07/NoiseDetectionCNN-GRU." May 11, 2024. Accessed: Jun. 01, 2024. [Online]. Available: https://github.com/xnejed07/NoiseDetectionCNN-GRU
[57] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic Controlled Averaging for Federated Learning." arXiv, Apr. 09, 2021. doi: 10.48550/arXiv.1910.06378.